MDM Migration Readiness Assessment & Wave Planning

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — MDM Migration Readiness Assessment & Wave Planning 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — MDM Migration Readiness Assessment & Wave Planning 5

What this solution is . . . 5

Why this matters (the why lives here, only here) . . . 5

How it works . . . 5

What you inventory and score . . . 6

The named artifacts (use these exact names everywhere) . . . 6

What this is not . . . 6

The standard deployment model . . . 7

2. Requirements Guide — MDM Migration Readiness Assessment & Wave Planning 7

2.1 Read access to the current (source) MDM . . . 7

2.2 An inventory export method (per source MDM) . . . 7

2.3 (Optional) Microsoft Graph access — when the current or target MDM is Intune-adjacent . . . 7

2.4 Target-platform context (what you are migrating to) . . . 8

2.5 A workstation to run the assessment scripts . . . 8

2.6 Stakeholder input (owners, criticality, change windows) . . . 8

2.7 Defined scope boundary . . . 8

Checklist . . . 8

3. Implementation Guide — MDM Migration Readiness Assessment & Wave Planning 9

The input schema (map your source export to these columns) . . . 9

Stage 1 — Assess (inventory + score) . . . 9

Stage 2 — Configure (design the waves) . . . 11

Stage 3 — Pilot . . . 12

Stage 4 — Validate (the gate) . . . 12

Stage 5 — Expand . . . 12

5. Testing Guide — MDM Migration Readiness Assessment & Wave Planning 13

Test 1 — Inventory completeness . . . 13

Test 2 — Inventory matches the source . . . 13

Test 3 — Readiness scorecard is populated and sane . . . 13

Test 4 — Risk register flags the real risks . . . 13

Test 5 — Dependencies and waves are coherent . . . 13

Test 6 — Cohort groups built correctly . . . 13
Test 7 — Plan validated against the pilot wave (the gate) . . . 13
Pass/fail gate . . . 14
6. Troubleshooting Guide — MDM Migration Readiness Assessment & Wave Planning 14
A. Inventory problems . . . 14
B. Microsoft Graph (optional, Intune-adjacent) issues . . . 15
C. Scoring looks wrong . . . 15
D. Wave / cohort problems . . . 15
E. Pilot validation gate . . . 15
F. Diagnostics quick reference . . . 16
G. When to escalate . . . 16
7. Best Practices — MDM Migration Readiness Assessment & Wave Planning 16
8. Deliverables Summary — MDM Migration Readiness Assessment & Wave Planning 17
Documentation . . . 17
PowerShell scripts . . . 17
Named artifacts produced . . . 18
What the buyer receives . . . 18
Outcome . . . 18
Appendix A — PowerShell Scripts 20

Executive Summary — MDM Migration Readiness Assessment & Wave Planning

Migrations · Vendor-neutral pre-migration assessment

Most MDM migrations fail in planning, not execution. Nobody has a clean inventory, the waves aren't thought through, and the risks surface mid-cutover during a change window with users watching. This solution is the planning discipline that runs **before** you touch a single device: a vendor-neutral **MDM migration readiness assessment and wave plan** — inventory the current estate, score readiness and risk, map dependencies, and produce a sequenced wave plan with explicit entry and exit criteria, validated against a pilot wave before it scales.

It is source-agnostic: it reads generic CSV exports from any MDM (Workspace ONE, MobileIron/Ivanti, Jamf, ManageEngine, SOTI, Configuration Manager, Intune, and others) and adds optional Microsoft Graph discovery when the current or target platform is Intune-adjacent. It plans the migration; it does not perform the cutover.

What you get

• A plain-language **Solution Overview** of the readiness-assessment model, what to inventory, and how the wave plan is built.

• A **Requirements Guide** with verify / implement-if-missing for every prerequisite.

• An **Implementation Guide** of precise, ordered steps — inventory, validate completeness, fill gaps, score readiness/risk, map dependencies, design waves, build cohorts, pilot, validate, expand.

• Five **PowerShell scripts** for estate-inventory discovery, completeness validation, a readiness/risk and wave-plan report, safe inventory-gap remediation, and a wave/cohort group builder.

• **Testing** and **Troubleshooting** guides with explicit pass/fail gates.

What this delivers

A complete, named set of planning artifacts — an `Estate Inventory`, a `Readiness Scorecard`, a `Risk Register`, a `Dependency Map`, a sequenced `Wave Plan` with `Wave Cohorts`, and per-wave `Entry/Exit Criteria` — so the actual migration runs to a script: lowest-risk, highest-readiness devices first, complex and critical devices last, and the plan proven on a pilot wave before any broad expansion.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with a mandatory gate to validate the plan against a pilot wave before it scales.

Outcome

A clear-eyed picture of exactly what you are migrating and in what order — inventoried, scored, and sequenced — so the migration runs to a script instead of by surprise.

1. Solution Overview — MDM Migration Readiness Assessment & Wave Planning

What this solution is

This is a **vendor-neutral, pre-migration assessment and wave plan** for moving a managed device estate from one Mobile Device Management (MDM) platform to another — for example Workspace ONE, MobileIron/Ivanti, Jamf, Kandji, ManageEngine, SOTI, Configuration Manager, or Group Policy moving to Microsoft Intune. It is the *planning* discipline that runs **before** a single device is touched: inventory the estate, score readiness and risk, find dependencies and blockers, then design a sequenced set of migration waves with explicit entry and exit criteria.

It does not perform the cutover. It produces the plan the cutover runs to.

Why this matters (the why lives here, only here)

Most MDM migrations fail in planning, not execution. The recurring pattern is the same every time:

- **No clean inventory.** Nobody can say exactly how many devices, which OS/platform mix, which apps, which configuration profiles, which enrollment types, or which dependencies are in scope.
- **Waves aren't thought through.** Devices are moved in an ad-hoc order, so high-risk or high-dependency devices migrate before anyone has validated the approach on safe ones.
- **Risks surface mid-cutover.** A blocker that should have been a line in a risk register on day one instead becomes an outage during a change window with users watching.

The repeatable way fixes this by front-loading the discovery and the decision-making. When the estate is inventoried, readiness and risk are scored, and the waves are sequenced with entry/exit gates, the actual migration runs to a script instead of by surprise — and it is validated against a pilot wave before it scales.

How it works

1. **Inventory the estate.** Collect every in-scope device with its OS/platform, ownership and enrollment type, installed/required apps, assigned configuration profiles, compliance state, and the user/department/region it belongs to — from the current MDM and from generic CSV inputs for any source platform. The artifact is the `Estate Inventory – MDM Migration – Assess`.
1. **Score readiness.** Each device is scored against objective readiness criteria (supported OS, reachable, enrolled, app coverage understood, no unsupported profile). The artifact is the `Readiness Scorecard – MDM Migration – Assess`.
1. **Score risk.** Each device/app/profile is scored for migration risk (executive or shared/critical device, kiosk/single-purpose, certificate or VPN dependency, unsupported configuration). The artifact is the `Risk Register – MDM Migration – Assess`.
1. **Map dependencies and blockers.** Identify what must move together or in a fixed order —

certificates, Wi-Fi/VPN profiles, line-of-business apps, conditional-access bindings, shared-device relationships. The artifact is the `Dependency Map – MDM Migration – Assess`.

1. **Design the waves.** Sequence devices into ordered waves from lowest risk/highest readiness to highest risk/lowest readiness, each with entry and exit criteria. The artifacts are the `Wave Plan – MDM Migration – Configure` and the `Wave Cohorts – MDM Migration – Configure`.
1. **Validate against a pilot.** The first wave is the `Pilot Wave – MDM Migration – Pilot`; it proves

the plan and the entry/exit gates before any broad expansion.

What you inventory and score

- **Devices** — count, hardware, ownership (corporate/BYOD), last check-in.
- **OS / platform mix** — Windows / iOS / iPadOS / macOS / Android and version spread.
- **Apps** — installed and required apps per device, including line-of-business and store apps.
- **Configuration profiles** — Wi-Fi, VPN, email, certificates, restrictions, kiosk/single-purpose.
- **Compliance** — current compliant/non-compliant state and the policy that sets it.
- **Enrollment types** — corporate ADE/automatic, user-driven, BYOD, shared/kiosk.
- **Dependencies** — certificate authorities, VPN, app dependencies, conditional access, shared-device.

The named artifacts (use these exact names everywhere)

Artifact	Stage
`Estate Inventory – MDM Migration – Assess`	Assess
`Readiness Scorecard – MDM Migration – Assess`	Assess
`Risk Register – MDM Migration – Assess`	Assess
`Dependency Map – MDM Migration – Assess`	Assess
`Wave Plan – MDM Migration – Configure`	Configure
`Wave Cohorts – MDM Migration – Configure`	Configure
`Cohort Group – Per Wave – Configure`	Configure
`Pilot Wave – MDM Migration – Pilot`	Pilot
`Entry/Exit Criteria – Per Wave – Pilot`	Pilot

What this is not

- It is **not** the cutover itself — no devices are re-enrolled or unenrolled by this solution.
- It is **not** vendor-specific tooling — it stays neutral about the source MDM and reads generic CSV/inventory exports so it works regardless of where you are migrating from.

• It does **not** assume the target is Intune, though the optional Microsoft Graph helpers light up when the current or target MDM is Intune-adjacent.

The standard deployment model

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand.*** *Inventory and score (Assess), build the wave plan and cohorts (Configure), run the first wave (Pilot), prove the entry/exit gates and the plan against that pilot (Validate), then sequence the remaining waves (Expand). The plan is always validated against a pilot wave before it scales — that gate is non-negotiable.*

2. Requirements Guide — MDM Migration Readiness Assessment & Wave Planning

Verify **every** requirement before assessing. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (target tenant, target licensing, source access, inventory, enrollment, pilot/wave structure, change control) are in `global-requirements-migrations.md` and are assumed; the items below are specific to running this assessment and wave-planning work.

2.1 Read access to the current (source) MDM

Description: Administrative read access to the MDM you are migrating *from* (Workspace ONE, MobileIron/Ivanti, Jamf, ManageEngine, SOTI, Configuration Manager, Intune, etc.). **Why:** The estate is inventoried from the source console; without read access you cannot enumerate devices, apps, profiles, compliance, or enrollment types. **Verify:** You can sign in to the source console and open the device, app, and profile lists. **Implement-if-missing:** Request a read-only admin role / API credential from the source platform owner; this assessment never requires write access to the source.

2.2 An inventory export method (per source MDM)

Description: A way to export the source estate to CSV — a console export, a reporting view, or an API pull — covering devices, OS/platform, apps, profiles, compliance, enrollment type, and owner. **Why:** The assessment scripts read generic CSV/inventory inputs so they stay vendor-neutral; you need a repeatable export to feed them. **Verify:** You can produce a CSV of in-scope devices with at least device name, OS/platform, owner, and enrollment type columns. **Implement-if-missing:** Use the source console's export/report feature, or its API, to produce the CSV. Map its columns to the input schema documented in the Implementation Guide.

2.3 (Optional) Microsoft Graph access — when the current or target MDM is Intune-adjacent

Description: Microsoft Graph PowerShell SDK and read scopes (`DeviceManagementManagedDevices.Read.All`, `DeviceManagementConfiguration.Read.All`, `DeviceManagementApps.Read.All`) where Intune is the source *or* the target. **Why:** When Intune is in the picture, the discovery script can pull a live, structured inventory directly instead of relying only on CSV exports. **Verify:** `Connect-MgGraph` succeeds

with the read scopes; `Get-MgDeviceManagementManagedDevice` returns devices.
Implement-if-missing: `Install-Module Microsoft.Graph -Scope CurrentUser`; consent to the read scopes. This is optional — the assessment runs CSV-only for any non-Intune source.

2.4 Target-platform context (what you are migrating to)

Description: A documented statement of the target platform, its supported OS baselines, and its enrollment channels for each in-scope platform. **Why:** Readiness is scored relative to the target — a device on an OS the target no longer supports is a readiness gap, not just an inventory line. **Verify:** You can name the target platform and its minimum supported OS per platform in scope.
Implement-if-missing: Capture the target's supported-OS matrix and enrollment prerequisites (see `global-requirements-migrations.md` G2 and G5) before scoring readiness.

2.5 A workstation to run the assessment scripts

Description: A Windows workstation with PowerShell 5.1 or PowerShell 7, able to reach the source export and (optionally) Microsoft Graph, with write access to an output folder for the artifacts. **Why:** The five PowerShell scripts generate the inventory, validation, report, and cohort artifacts to disk. **Verify:** `\$PSVersionTable.PSVersion` shows 5.1+ or 7+; you can write to the chosen output folder.
Implement-if-missing: Use any managed admin workstation; create an output folder (e.g. `C:\MDMRDY`) and run the scripts from the `scripts` folder of this package.

2.6 Stakeholder input (owners, criticality, change windows)

Description: Business input on which users/departments/devices are executive, shared, kiosk, or mission-critical, and any blackout/change-window constraints. **Why:** Risk scoring and wave sequencing depend on business criticality and change windows that no inventory export contains. **Verify:** You have a named contact per department/region who can confirm criticality and windows. **Implement-if-missing:** Run a short stakeholder intake; record criticality and constraints as input columns so they feed the `Risk Register – MDM Migration – Assess` and the `Wave Plan – MDM Migration – Configure`.

2.7 Defined scope boundary

Description: A written statement of what is in and out of scope (sites, platforms, ownership types, retired devices). **Why:** An unbounded inventory inflates risk and waves with devices that should be excluded or decommissioned, not migrated. **Verify:** The scope statement names included platforms/sites and explicitly excludes retired or out-of-scope devices. **Implement-if-missing:** Agree the scope boundary with the migration sponsor before inventory; encode exclusions as a filter on the CSV input.

Checklist

- Read access to the current (source) MDM
- An inventory export method that produces CSV (devices, OS, apps, profiles, compliance, enrollment, owner)
- (Optional) Microsoft Graph read access when Intune is source or target
- Target-platform context (supported OS baselines + enrollment channels)

- A workstation with PowerShell 5.1/7 and an output folder
- Stakeholder input on criticality and change windows
- A written scope boundary (in/out of scope)
- Category global requirements confirmed (`global-requirements-migrations.md`)

3. Implementation Guide — MDM Migration Readiness Assessment & Wave Planning

Follow these steps **in order**. This is a planning engagement: the output is a set of named artifacts, not a changed device. The model is **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a hard gate that the plan is **validated against a pilot wave** before it scales.

> ***Vendor-neutral note:*** *every step reads either a generic CSV export from the source MDM or, when the current/target MDM is Intune-adjacent, a live Microsoft Graph pull. No step requires write access to the source platform.*

The input schema (map your source export to these columns)

The scripts read a CSV (the raw estate export) with these columns. Map your source MDM's export columns to these names, or let `MDMRDY-Discovery.ps1` build them from Microsoft Graph.

Column	Meaning
`DeviceName`	Unique device identifier/name
`Platform`	`Windows` / `iOS` / `iPadOS` / `macOS` / `Android`
`OSVersion`	Current OS version string
`Owner`	User principal name or owner identifier
`Department`	Department/business unit
`Region`	Site/region
`EnrollmentType`	`Corporate-ADE` / `UserDriven` / `BYOD` / `Shared` / `Kiosk`
`Compliance`	`Compliant` / `NonCompliant` / `Unknown`
`LastCheckIn`	Last check-in date (ISO 8601)
`Apps`	Semicolon-separated app list (or app count)
`Profiles`	Semicolon-separated profile list (Wi-Fi;VPN;Cert;Email;Restrictions;Kiosk)
`Criticality`	`Standard` / `Executive` / `Critical` (from stakeholder input)

Stage 1 — Assess (inventory + score)

Step 1 — Inventory the estate

1. Produce the raw export from the source MDM (CSV), or confirm Microsoft Graph read access if Intune

is source/target.

1. From the `scripts` folder run the discovery (read-only):

• CSV source: `powershell -ExecutionPolicy Bypass -File .\MDMRDY-Discovery.ps1 -InputCsv .\source-export.csv -OutputFolder C:\MDMRDY`

• Intune source: `powershell -ExecutionPolicy Bypass -File .\MDMRDY-Discovery.ps1 -UseGraph -OutputFolder C:\MDMRDY`

1. Output: the `Estate Inventory – MDM Migration – Assess` (a normalized CSV of every in-scope device

with platform, OS, owner, department/region, enrollment type, compliance, apps, and profiles).

Step 2 — Validate inventory completeness

1. Run the validation (read-only) against the inventory:

```
powershell -ExecutionPolicy Bypass -File .\MDMRDY-Validation.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY
```

1. It flags missing/blank fields, unknown platforms, stale check-ins, and rows that cannot be scored.

2. Output: a data-completeness report. Do not score readiness until completeness passes the gate in

the Testing Guide (Test 1).

Step 3 — Fill inventory gaps (safe remediation)

1. Where rows are incomplete, run the remediation in its **default** `-WhatIf` **mode** first to preview

the fills: `powershell -ExecutionPolicy Bypass -File .\MDMRDY-Remediation.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY`

1. Review the proposed fills (defaulting unknown enrollment type, inferring platform from OS, marking

stale devices for review). Re-run with `-Apply` only when the proposed changes are correct.

1. Output: a completed `Estate Inventory – MDM Migration – Assess` ready to score.

Step 4 — Score readiness and risk, map dependencies

1. Run the report (read-only) over the completed inventory:

```
powershell -ExecutionPolicy Bypass -File .\MDMRDY-Report.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY
```

1. It produces, in one pass:

• the `Readiness Scorecard – MDM Migration – Assess` (per-device readiness score 0–100 against

supported OS, reachable/recent check-in, enrolled, app coverage understood, no unsupported profile),

• the `Risk Register – MDM Migration – Assess` (per-device risk score and the contributing risk

factors: executive/critical, shared/kiosk, certificate/VPN dependency, unsupported config, non-compliant, stale),

• the `Dependency Map – MDM Migration – Assess` (what must move together/in order: certificates,

Wi-Fi/VPN, line-of-business apps, conditional access, shared-device groupings),

• and a **draft** `Wave Plan – MDM Migration – Configure` with suggested wave assignment.

Stage 2 — Configure (design the waves)

Step 5 — Finalize the wave plan and cohorts

1. Review the draft wave plan with stakeholders; adjust assignments for criticality and change windows that only the business knows.

1. The wave model sequences from **lowest risk / highest readiness** to **highest risk / lowest readiness**:

• **Wave 0 — Pilot:** a small, low-risk, high-readiness, representative cross-section.

• **Wave 1 — Early:** standard devices, no special dependencies.

• **Wave 2 — Broad:** the bulk of standard devices.

• **Wave 3 — Complex/Critical:** executive, shared/kiosk, certificate/VPN-dependent, low-readiness.

1. Each wave carries an `Entry/Exit Criteria – Per Wave – Pilot` block (see Step 6).

2. Output: the finalized `Wave Plan – MDM Migration – Configure` and the `Wave Cohorts – MDM Migration – Configure` (the per-wave device membership lists).

Step 6 — Define entry/exit criteria per wave

Record, for every wave, the `Entry/Exit Criteria – Per Wave – Pilot`:

• **Entry:** prior wave exited cleanly; target enrollment channel active for the platforms in this wave (`global-requirements-migrations.md` G5); rollback path documented (G7); cohort group built.

• **Exit:** all devices in the cohort migrated and enrolled in the target; compliance re-established; no open severity-1 issue; user-impact tickets below the agreed threshold.

Step 7 — Build the cohort groups

1. Materialize each wave's membership as a `Cohort Group – Per Wave – Configure`.

2. Build the groups from the cohort lists:

• Export-only (any MDM): `powershell -ExecutionPolicy Bypass -File .\MDMRDY-Set-WaveCohortTargeting.ps1 -CohortsCsv C:\MDMRDY\wave-cohorts.csv -OutputFolder C:\MDMRDY` — writes per-wave membership CSVs you can import into the target's grouping system.

• Entra/Intune target: add `-CreateEntraGroups -WhatIf` to preview, then `-CreateEntraGroups -Apply` to create one Microsoft Entra security group per wave and add members (requires Graph `Group.ReadWrite.All`).

1. Naming: each group is `Cohort Group – Per Wave – Configure`, suffixed with the wave label (e.g.

`MDMRDY-Wave0-Pilot`, `MDMRDY-Wave1-Early`).

Stage 3 — Pilot

Step 8 — Run the pilot wave

1. The `Pilot Wave – MDM Migration – Pilot` (Wave 0) is the first cohort to migrate, using your

separate migration runbook (this solution plans; it does not cut over).

1. Confirm the pilot meets its `Entry/Exit Criteria – Per Wave – Pilot` before starting.

Stage 4 — Validate (the gate)

Step 9 — Validate the plan against the pilot

1. After the pilot wave migrates, re-run `MDMRDY-Discovery.ps1` and `MDMRDY-Report.ps1` against the

post-pilot estate to confirm the pilot's exit criteria are met and the readiness/risk model held.

1. Reconcile predicted vs. actual: did the readiness scores predict the smooth devices, and did the

risk register predict the friction? Adjust the scoring weights and the remaining waves if not.

1. **Do not expand** until the pilot wave passes its exit criteria and the plan is validated. This is

the mandatory gate.

Stage 5 — Expand

Step 10 — Sequence the remaining waves

1. With the plan validated, release the remaining waves in order, each gated by its

`Entry/Exit Criteria – Per Wave – Pilot`.

1. Re-run `MDMRDY-Report.ps1` after each wave to keep the readiness/risk picture and the cohorts

current as devices leave the source estate.

Order of operations

`Inventory (Discovery) → Validate completeness → Fill gaps (Remediation, -WhatIf first) → Score readiness/risk + dependencies (Report) → Finalize Wave Plan + Cohorts → Entry/Exit criteria → Build cohort groups → Pilot wave → Validate plan against pilot (gate) → Expand remaining waves`

5. Testing Guide — MDM Migration Readiness Assessment & Wave Planning

Run these tests as you produce each artifact. The whole point is that the plan is **validated against a pilot wave** before it scales (Tests 6–7), so no artifact is "done" until its gate passes.

Test 1 — Inventory completeness

Do: Run `MDMRDY-Validation.ps1` against the `Estate Inventory – MDM Migration – Assess`. **Expected:** No blank required fields (`DeviceName`, `Platform`, `OSVersion`, `EnrollmentType`, `Owner`); no unknown platforms; stale check-ins flagged for review. Completeness at or above the agreed threshold.

Test 2 — Inventory matches the source

Do: Compare the inventory device count and platform mix against the source MDM console totals. **Expected:** Counts reconcile within the agreed tolerance; any difference is explained by the scope boundary (excluded/retired devices).

Test 3 — Readiness scorecard is populated and sane

Do: Open the `Readiness Scorecard – MDM Migration – Assess`. **Expected:** Every in-scope device has a readiness score 0–100; devices on unsupported OS or with unknown app/profile coverage score low; recent, supported, enrolled devices score high.

Test 4 — Risk register flags the real risks

Do: Open the `Risk Register – MDM Migration – Assess`. **Expected:** Executive/critical, shared/kiosk, certificate/VPN-dependent, non-compliant, and stale devices carry elevated risk with the contributing factors named; standard devices score low.

Test 5 — Dependencies and waves are coherent

Do: Cross-check the `Dependency Map – MDM Migration – Assess` against the `Wave Plan – MDM Migration – Configure` and `Wave Cohorts – MDM Migration – Configure`. **Expected:** Devices that must move together share a wave; high-risk/low-readiness devices land in later waves; the `Pilot Wave – MDM Migration – Pilot` is low-risk, high-readiness, and representative; every device appears in exactly one wave.

Test 6 — Cohort groups built correctly

Do: Run `MDMRDY-Set-WaveCohortTargeting.ps1`; inspect the per-wave membership output (and, if created, the Entra security groups). **Expected:** Each `Cohort Group – Per Wave – Configure` contains exactly its cohort's devices; no device is in two groups; group names match the wave labels.

Test 7 — Plan validated against the pilot wave (the gate)

Do: After the pilot migrates, re-run `MDMRDY-Discovery.ps1` and `MDMRDY-Report.ps1` over the post-pilot estate and check the pilot's `Entry/Exit Criteria – Per Wave – Pilot`. **Expected:** Pilot devices migrated, enrolled in the target, and compliant; predicted readiness/risk matched the actual outcome; no open severity-1 issue. Scoring weights tuned if prediction missed.

Pass/fail gate

Gate	Requirement
Assess pass	Tests 1–4 pass; inventory reconciles to source; readiness and risk scored for every in-scope device
Configure pass	Test 5–6 pass; every device in exactly one wave; pilot wave is low-risk/high-readiness/representative; entry/exit criteria written per wave
Pilot/Validate pass	Test 7 passes; pilot wave meets its exit criteria and the readiness/risk model is confirmed before any expansion

6. Troubleshooting Guide — MDM Migration Readiness Assessment & Wave Planning

Use `MDMRDY-Validation.ps1` for inventory data quality, `MDMRDY-Report.ps1` to re-score, and `MDMRDY-Remediation.ps1` (default `-WhatIf`) to safely fill gaps. All discovery/validation/report steps are read-only.

A. Inventory problems

Symptom	Cause	Remediation
Inventory count doesn't match the source console	Scope filter too wide/narrow, or stale/retired devices included	Reconcile against the scope boundary; exclude retired devices; re-run `MDMRDY-Discovery.ps1`
Blank required fields (`Platform`, `EnrollmentType`, `Owner`)	Source export omitted columns or used different headers	Map source columns to the input schema (Implementation Guide); run `MDMRDY-Remediation.ps1 -WhatIf` then `-Apply` to fill defaults
Unknown platform values	Source uses non-standard platform labels	Normalize to `Windows/iOS/iPadOS/macOS/Android`; remediation infers platform from `OSVersion` where possible
Devices with very old `LastCheckIn`	Decommissioned or lost devices still in source	Flag stale in validation; exclude from waves or route to a decommission list, not a migration wave

B. Microsoft Graph (optional, Intune-adjacent) issues

Symptom	Cause	Remediation
`Connect-MgGraph` fails	SDK missing or scopes not consented	`Install-Module Microsoft.Graph -Scope CurrentUser`; consent to the read scopes in Requirements 2.3
Graph returns no devices	Wrong tenant or insufficient role	Confirm you signed into the correct tenant with a role that can read managed devices
You don't use Intune at all	Source is a non-Intune MDM	Skip `-UseGraph`; run CSV-only — the assessment is fully vendor-neutral without Graph

C. Scoring looks wrong

Symptom	Cause	Remediation
Everything scores low-readiness	Target supported-OS baseline not provided	Supply the target's supported-OS matrix; re-run `MDMRDY-Report.ps1`
Critical devices score low-risk	Stakeholder criticality not fed in	Populate the `Criticality` column from stakeholder input; re-run the report
Readiness and actual pilot outcome disagree	Scoring weights mis-tuned	Reconcile predicted vs. actual after the pilot; adjust weights and re-score the remaining waves

D. Wave / cohort problems

Symptom	Cause	Remediation
A device appears in two waves	Overlapping cohort filters	Re-run `MDMRDY-Set-WaveCohortTargeting.ps1`; ensure each device maps to exactly one wave
Dependent devices split across waves	Dependency map not applied to sequencing	Use the `Dependency Map – MDM Migration – Assess` to co-locate dependent devices in one wave
Entra group creation denied	Missing `Group.ReadWrite.All`	Grant the write scope, or use export-only mode and import the membership CSVs into the target manually

E. Pilot validation gate

- If the `Pilot Wave – MDM Migration – Pilot` does not meet its `Entry/Exit Criteria – Per Wave – Pilot`,

do not expand. Resolve the failing criterion, re-run discovery + report over the post-pilot estate, and only proceed once the exit criteria pass and the model is confirmed.

F. Diagnostics quick reference

```
# Re-check inventory completeness
.\MDMRDY-Validation.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY
# Preview gap fills without changing the source of truth
.\MDMRDY-Remediation.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY   # -WhatIf
# Re-score after fixing inputs
.\MDMRDY-Report.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY
```

G. When to escalate

If the inventory reconciles to the source, completeness passes, the target baseline and criticality are supplied, and the readiness/risk scores still do not match what the pilot wave actually experienced, pause the expansion and review the scoring model and the dependency map with the migration sponsor before sequencing further waves.

7. Best Practices — MDM Migration Readiness Assessment & Wave Planning

- **Inventory before you opinion.** Build the `Estate Inventory – MDM Migration – Assess` first and

reconcile it to the source console totals. Every later decision (readiness, risk, waves) is only as good as this inventory — a migration planned on a guess fails on a guess.

- **Score against the target, not in a vacuum.** Readiness is relative: a device on an OS the *target*

no longer supports is a readiness gap. Capture the target's supported-OS matrix before scoring the `Readiness Scorecard – MDM Migration – Assess`.

- **Make business criticality an input, not an afterthought.** No inventory export knows which devices

are executive, shared, or mission-critical. Feed stakeholder criticality into the `Risk Register – MDM Migration – Assess` so the risky devices land in later waves.

- **Sequence low-risk/high-readiness first.** Always migrate the safest, most-ready devices early so

the approach is proven before complex or critical devices move. The `Pilot Wave – MDM Migration – Pilot` must be low-risk, high-readiness, and representative.

- **Respect dependencies — move what must move together.** Use the

`Dependency Map – MDM Migration – Assess` to keep certificate/VPN/app/conditional-access and shared-device groupings in the same wave; a split dependency is an outage waiting for a change window.

- **Write entry and exit criteria for every wave.** A wave without an

`Entry/Exit Criteria – Per Wave – Pilot` block has no gate, so it never truly "finishes" and risk piles into the next wave. Define both before the wave starts.

• **Validate the plan against the pilot before you scale.** Re-run the report over the post-pilot

estate and reconcile predicted vs. actual; tune the scoring weights if the pilot surprised you. Do not expand on an unvalidated plan — this is the one gate you never skip.

• **Keep it vendor-neutral and repeatable.** Drive everything from the CSV input schema (or Graph

when Intune-adjacent) so the same assessment runs against any source MDM and the next migration reuses the artifacts and names verbatim.

• **Re-run discovery as the estate drains.** After each wave, re-run `MDMRDY-Discovery.ps1` and

`MDMRDY-Report.ps1` so the inventory, scores, and cohorts stay current as devices leave the source.

• **Preview before you change anything.** Run `MDMRDY-Remediation.ps1` in its default `-WhatIf` mode

and review the proposed inventory fills before `-Apply`; never let an automated fill silently rewrite the source of truth.

• **Name everything once and reuse it.** Use the exact artifact names

(`<Purpose> – <Scope> – <Stage>`) across the inventory, scorecard, risk register, dependency map, wave plan, cohorts, and groups so the plan reads the same in every document and tool.

8. Deliverables Summary — MDM Migration Readiness Assessment & Wave Planning

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What the assessment is, what to inventory, readiness/risk scoring, wave model, and the why
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Migrations)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: inventory, validate, fill gaps, score, design waves, build cohorts, pilot, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`MDMRDY-Discovery.ps1`	Builds the `Estate Inventory – MDM Migration – Assess` from a source CSV export or (optional) Microsoft Graph. Read-only.
`MDMRDY-Validation.ps1`	Checks inventory data completeness (missing fields, unknown platforms, stale check-ins). Read-only.
`MDMRDY-Report.ps1`	Produces the `Readiness Scorecard`, `Risk Register`, `Dependency Map`, and a draft `Wave Plan` in one pass. Read-only.
`MDMRDY-Remediation.ps1`	Safely fills inventory gaps; default `-WhatIf`, applies only with `-Apply`.
`MDMRDY-Set-WaveCohortTargeting.ps1`	Builds the `Wave Cohorts` / `Cohort Group – Per Wave – Configure` membership (export CSV, or optional Entra security groups).

Named artifacts produced

Artifact	Stage
`Estate Inventory – MDM Migration – Assess`	Assess
`Readiness Scorecard – MDM Migration – Assess`	Assess
`Risk Register – MDM Migration – Assess`	Assess
`Dependency Map – MDM Migration – Assess`	Assess
`Wave Plan – MDM Migration – Configure`	Configure
`Wave Cohorts – MDM Migration – Configure`	Configure
`Cohort Group – Per Wave – Configure`	Configure
`Pilot Wave – MDM Migration – Pilot`	Pilot
`Entry/Exit Criteria – Per Wave – Pilot`	Pilot

What the buyer receives

- This assessment and wave-planning guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable, vendor-neutral process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

A clear-eyed picture of exactly what is being migrated and in what order — the estate inventoried, readiness and risk scored, dependencies mapped, and a sequenced wave plan with entry/exit criteria — validated against a pilot wave before it scales, so the migration runs to a script instead of by surprise.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MDMRDY-Discovery.ps1

Purpose: Produces the "Estate Inventory - MDM Migration - Assess" artifact: a normalized CSV of every in-scope device with platform, OS, owner, department/region, enrollment type, compliance, apps, and profiles. Reads a generic CSV export from ANY source MDM, or (optional) pulls a live inventory from Microsoft Graph when the current/target MDM is Intune-adjacent. Read-only: it never writes to the source platform. HOW TO RUN CSV source (any MDM): powershell -ExecutionPolicy Bypass -File .\MDMRDY-Discovery.ps1 -InputCsv .\source-export.csv -OutputFolder C:\MDMRDY Intune source/target (optional): powershell -ExecutionPolicy Bypass -File .\MDMRDY-Discovery.ps1 -UseGraph -OutputFolder C:\MDMRDY Graph mode needs Microsoft.Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS Writes estate-inventory.csv (the "Estate Inventory - MDM Migration - Assess") to the output folder and prints a device/platform summary. No source-side changes. .NOTES Migrations — vendor-neutral pre-migration assessment. Read-only. #> [CmdletBinding()] param([string]$InputCsv, [switch]$UseGraph, [string]$OutputFolder = (Join-Path $PWD 'MDMRDY'))

How to run: CSV source (any MDM): powershell -ExecutionPolicy Bypass -File .\MDMRDY-Discovery.ps1 -InputCsv .\source-export.csv -OutputFolder C:\MDMRDY Intune source/target (optional): powershell -ExecutionPolicy Bypass -File .\MDMRDY-Discovery.ps1 -UseGraph -OutputFolder C:\MDMRDY Graph mode needs Microsoft.Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS Writes estate-inventory.csv (the "Estate Inventory - MDM Migration - Assess") to the output folder and prints a device/platform summary. No source-side changes. .NOTES Migrations — vendor-neutral pre-migration assessment. Read-only. #> [CmdletBinding()] param([string]$InputCsv, [switch]$UseGraph, [string]$OutputFolder = (Join-Path $PWD 'MDMRDY'))

MDMRDY-Remediation.ps1

Purpose: Fills gaps in the "Estate Inventory - MDM Migration - Assess" so it can be scored: defaults an unknown EnrollmentType to 'UserDriven', infers Platform from OSVersion where blank, and marks stale or unscoreable rows for review. SAFE BY DEFAULT: previews changes with -WhatIf and writes nothing until you pass -Apply. It never touches the source MDM — only the local inventory CSV. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\MDMRDY-Remediation.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY Apply (writes a remediated copy): powershell -ExecutionPolicy Bypass -File .\MDMRDY-Remediation.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY -Apply EXPECTED RESULTS -WhatIf: prints each proposed fill, writes nothing. -Apply : writes estate-inventory-remediated.csv with the fills applied; original is untouched. .NOTES Migrations — vendor-neutral pre-migration assessment. Default -WhatIf; -Apply to change the copy. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$OutputFolder = (Join-Path $PWD 'MDMRDY'), [switch]$Apply)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\MDMRDY-Remediation.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY Apply (writes a remediated copy): powershell -ExecutionPolicy Bypass -File .\MDMRDY-Remediation.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY -Apply EXPECTED RESULTS -WhatIf:

prints each proposed fill, writes nothing. -Apply : writes estate-inventory-remediated.csv with the fills applied; original is untouched. .NOTES Migrations — vendor-neutral pre-migration assessment. Default -WhatIf; -Apply to change the copy. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$OutputFolder = (Join-Path $PWD 'MDMRDY'), [switch]$Apply)

MDMRDY-Report.ps1

Purpose: From the "Estate Inventory - MDM Migration - Assess", produces in one pass: - "Readiness Scorecard - MDM Migration - Assess" (per-device readiness 0-100) - "Risk Register - MDM Migration - Assess" (per-device risk + factors) - "Dependency Map - MDM Migration - Assess" (what must move together) - draft "Wave Plan - MDM Migration - Configure" (suggested wave per device) Read-only: it computes scores from the inventory and writes report CSVs only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDMRDY-Report.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY Optional: -SupportedOS @{ Windows='10.0.19045'; iOS='16'; iPadOS='16'; macOS='13'; Android='12' } -StaleDays 90 EXPECTED RESULTS Writes readiness-scorecard.csv, risk-register.csv, dependency-map.csv, and wave-plan-draft.csv, then prints a readiness/risk and wave-distribution summary. .NOTES Migrations — vendor-neutral pre-migration assessment. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$OutputFolder = (Join-Path $PWD 'MDMRDY'), [hashtable]$SupportedOS = @{ Windows='10.0.19044'; iOS='16'; iPadOS='16'; macOS='13'; Android='11' }, [int]$StaleDays = 90)

How to run: powershell -ExecutionPolicy Bypass -File .\MDMRDY-Report.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY Optional: -SupportedOS @{ Windows='10.0.19045'; iOS='16'; iPadOS='16'; macOS='13'; Android='12' } -StaleDays 90 EXPECTED RESULTS Writes readiness-scorecard.csv, risk-register.csv, dependency-map.csv, and wave-plan-draft.csv, then prints a readiness/risk and wave-distribution summary. .NOTES Migrations — vendor-neutral pre-migration assessment. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$OutputFolder = (Join-Path $PWD 'MDMRDY'), [hashtable]$SupportedOS = @{ Windows='10.0.19044'; iOS='16'; iPadOS='16'; macOS='13'; Android='11' }, [int]$StaleDays = 90)

MDMRDY-Set-WaveCohortTargeting.ps1

Purpose: Materializes the "Wave Cohorts - MDM Migration - Configure" from the finalized wave plan: one membership list per wave ("Cohort Group - Per Wave - Configure"). By default it writes export CSVs you can import into ANY target's grouping system (vendor-neutral). When the target is Entra/Intune, -CreateEntraGroups creates one Microsoft Entra security group per wave and adds members. Group creation is gated: -WhatIf previews, -Apply commits. HOW TO RUN Export-only (any MDM): powershell -ExecutionPolicy Bypass -File .\MDMRDY-Set-WaveCohortTargeting.ps1 -CohortsCsv C:\MDMRDY\wave-plan-draft.csv -OutputFolder C:\MDMRDY Entra/Intune target — preview then apply: ... -CreateEntraGroups -WhatIf ... -CreateEntraGroups -Apply Entra mode needs Microsoft.Graph SDK and Group.ReadWrite.All. The cohorts CSV needs at least: DeviceName, SuggestedWave (or WaveLabel). EXPECTED RESULTS Writes one cohort-.csv per wave (the "Cohort Group - Per Wave - Configure"). With -CreateEntraGroups -Apply, also creates Entra security groups named MDMRDY- and adds device members. -WhatIf changes nothing in Entra. .NOTES Migrations — vendor-neutral pre-migration assessment. Export-only is read-only; Entra writes only with -Apply. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$CohortsCsv, [string]$OutputFolder = (Join-Path $PWD 'MDMRDY'), [switch]$CreateEntraGroups, [switch]$Apply)

How to run: Export-only (any MDM): powershell -ExecutionPolicy Bypass -File .\MDMRDY-Set-WaveCohortTargeting.ps1 -CohortsCsv C:\MDMRDY\wave-plan-draft.csv -OutputFolder C:\MDMRDY Entra/Intune target — preview then apply: ... -CreateEntraGroups -WhatIf ... -CreateEntraGroups -Apply Entra mode needs Microsoft.Graph SDK and Group.ReadWrite.All. The cohorts CSV needs at least: DeviceName, SuggestedWave (or WaveLabel). EXPECTED RESULTS Writes one cohort-<WaveLabel>.csv per wave (the "Cohort Group - Per Wave - Configure"). With -CreateEntraGroups -Apply, also creates Entra security groups named MDMRDY-<WaveLabel> and adds device members. -WhatIf changes nothing in Entra. .NOTES Migrations — vendor-neutral pre-migration assessment. Export-only is read-only; Entra writes only with -Apply. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$CohortsCsv, [string]$OutputFolder = (Join-Path $PWD 'MDMRDY'), [switch]$CreateEntraGroups, [switch]$Apply)

MDMRDY-Validation.ps1

Purpose: Validates the "Estate Inventory - MDM Migration - Assess" for completeness: missing/blank required fields, unknown platform values, and stale check-ins. Gates the assessment so you do not score readiness/risk on bad data. Read-only — reports issues, changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDMRDY-Validation.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY Optional: -StaleDays 90 (default) sets the stale check-in threshold. EXPECTED RESULTS Prints a completeness summary and writes validation-report.csv listing every flagged row and reason. A clean run (no blanks, known platforms) means the inventory is ready to score. .NOTES Migrations — vendor-neutral pre-migration assessment. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$OutputFolder = (Join-Path $PWD 'MDMRDY'), [int]$StaleDays = 90)

How to run: powershell -ExecutionPolicy Bypass -File .\MDMRDY-Validation.ps1 -InventoryCsv C:\MDMRDY\estate-inventory.csv -OutputFolder C:\MDMRDY Optional: -StaleDays 90 (default) sets the stale check-in threshold. EXPECTED RESULTS Prints a completeness summary and writes validation-report.csv listing every flagged row and reason. A clean run (no blanks, known platforms) means the inventory is ready to score. .NOTES Migrations — vendor-neutral pre-migration assessment. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$OutputFolder = (Join-Path $PWD 'MDMRDY'), [int]$StaleDays = 90)